

SWEN90010 2022 Exam - Markdown Practice Version

Source PDF: materials/2022 - Exam.pdf

Visible total: 70 marks across 17 questions.

Extraction note: the source PDF includes Studocu watermarks and visible submitted-answer annotations. This practice version keeps the exam prompts and omits those submitted answers from the main question text. Dropdown-style Canvas questions have been converted into fill-in prompts or tables for practice.

Q1. Fault Trees - 5 marks

An electric saw employs an automatic stopping feature that comprises the following components:

- Three proximity sensors are employed to detect when an object is sufficiently close to the saw blade to constitute a hazard. Each sensor returns readings that indicate the distance from the saw blade to the closest object as measured in centimetres (cm), as detected by that sensor. These sensors can each fail independently, in which case they return incorrect readings.
- The safety controller takes input from the three proximity sensors. To calculate how far the closest object is to the saw blade, the safety controller takes the minimum value from the three sensor readings. The safety controller determines that an object is sufficiently close to the saw blade to constitute a hazard when this distance is equal to or below 10 cm.
- The brake actuator is activated by the safety controller to stop the saw blade from moving if an object is detected that is within 10 cm of the saw.

Draw a fault tree that describes the conditions under which the following hazard can occur:

An object is within 10cm of the saw blade but the brake actuator is not activated.

This hazard should be the root of your fault tree. The leaves should be the following primitive failure actions:

- There are three separate failure actions to represent failure of each of the three proximity sensors respectively.
- There is one primitive failure action to represent when the safety controller fails.

You may write your fault tree as preformatted text. For instance, to represent the fault tree with root A, and child nodes B and C, connected by an OR gate:

```
A
+ OR
  B
  C
```

If node B has children E and F connected by an AND gate:

```
A
+ OR
  B
  + AND
    E
    F
  C
```

Your answer:

Q1 Full-Mark Answer

```
Object is within 10cm of the saw blade but the brake actuator is not activated
+ OR
  safety controller failed
  safety controller does not detect that object is within 10cm
+ AND
  proximity sensor 1 failed
  proximity sensor 2 failed
  proximity sensor 3 failed
```

Reason: The safety controller activates the brake actuator when its computed distance is at most 10cm. Since it computes the distance by taking the minimum of the three sensor readings, one working sensor that reports a value at or below 10cm is enough to trigger the brake. Therefore, if the controller itself has not failed, the hazard can occur through sensor failure only when all three sensors fail in a way that prevents the controller from seeing any reading at or below 10cm.

Schema: With a minimum-of-readings design, the safety function is lost only if every redundant sensor fails to report the hazard. That gives an AND gate over the three sensor failures. Independent alternative causes, such as controller failure, are connected by OR.

Q2. Voting - 3 marks

Continuing the electric saw scenario from Q1.

Suppose the safety controller is modified so that it now behaves as follows. To calculate the distance from the saw blade to the closest object it now employs majority voting. The majority voting algorithm considers two sensor values sufficiently equal when the absolute value of the difference between them is less than 1 cm.

As before, if the majority voting algorithm determines that the distance from the saw blade to the nearest object is equal to or below 10 cm, it activates the brake actuator. If the majority voting algorithm produces no result, then the safety controller concludes that one of the sensors must have failed and so activates the brake actuator to stop the saw blade from moving.

For each scenario, determine what action the safety controller will take.

Scenario	Readings received	Action taken	Reason
1	{11, 10, 9}		
2	{10, 10, 11}		
3	{11, 11, 10}		

Q2 Full-Mark Answer

Scenario	Readings received	Action taken	Reason
1	{11, 10, 9}	Activate brake actuator	No two readings are sufficiently equal because each adjacent pair differs by exactly 1cm, and the rule requires difference less than 1cm. So majority voting produces no result, and the controller stops the saw as a fail-safe action.
2	{10, 10, 11}	Activate brake actuator	The two 10 readings form a majority value of 10cm. Since $10 \leq 10$, the controller detects a hazard and activates the brake.
3	{11, 11, 10}	Do not activate brake actuator	The two 11 readings form a majority value of 11cm. Since $11 > 10$, the controller does not detect a hazard.

Schema: First find whether there is a majority pair using the exact equality tolerance. If there is a majority result, compare that result with the hazard threshold. If there is no majority result, follow the fail-safe rule and activate the brake.

Q3. Alloy 1 - 2 marks

Consider an Alloy model of a simplified university records system. This system records information about two disjoint groups of people: staff and students. Each staff member is identified by a unique Staff ID. Likewise each student is identified by a unique Student ID. Staff and Student IDs are disjoint.

For each enrolled student, the system records which kind of degree the student is studying, and in this simplified model each student can be studying only one kind of degree at a time. There are three kinds of degrees: Undergraduate, Graduate Coursework, and Graduate Research.

Each student studying a Graduate Research degree has at least one supervisor. Supervisors are current staff members. Only students studying Graduate Research degrees have supervisors. The system records, for each staff member, the set of students they are supervising.

Each current staff member is assigned one of two roles: either Academic or Administrator. Only Academic staff members can supervise students.

The following Alloy signature declarations have been used to model the state of this system.

```
sig StudentID {}

abstract sig DegreeType {}
one sig Undergraduate, GraduateCoursework, GraduateResearch extends DegreeType {}

sig StaffID {}

abstract sig Role {}
one sig Academic, Administrator extends Role {}

one sig State {
  var staff_roles: StaffID -> lone Role,
  var supervising: StaffID -> set StudentID,
  var student_degrees: StudentID -> lone DegreeType
}
```

Explain why the multiplicity one has been used on the State signature, and the multiplicity lone has been used on each of the staff_roles and student_degrees fields. For instance, how would the model be different if each of these multiplicity keywords was absent?

Your answer:

Q3 Full-Mark Answer

`one sig State` means there is exactly one State object. This is appropriate because the model represents one current university-records state whose mutable fields change over time. If `one` were omitted, Alloy could allow zero or many State atoms, and expressions such as `State.staff\_roles` could mix information from multiple states.

`staff\_roles : StaffID -> lone Role` means each StaffID is mapped to at most one role. This matches the requirement that each current staff member has exactly one role, while a StaffID with no role can represent someone who is not current staff. If `lone` were omitted, a staff ID could be mapped to multiple roles, for example both Academic and Administrator.

`student\_degrees : StudentID -> lone DegreeType` means each StudentID is mapped to at most one degree type. This matches the requirement that each enrolled student studies only one degree type, while a StudentID with no degree can represent a student who is not enrolled. If `lone` were omitted, a student could be mapped to several degree types at the same time.

Schema: In Alloy, one `sig X` constrains the number of atoms in a signature. For relation fields, `A -> lone B` constrains each A atom to map to at most one B atom, while still allowing no mapping.

Q4. Alloy 2 - 3 marks

Continuing the university records scenario from Q3.

Write an Alloy function `supervisors` that takes a `StudentID` as its argument and returns a set of `StaffID` that are the staff members who are supervisors of the student identified by the given `StudentID`.

```
fun supervisors[s : StudentID] : set StaffID {
  // FILL IN HERE
}
```

Q4 Full-Mark Answer

```
fun supervisors[s : StudentID] : set StaffID {
  { staff : StaffID | s in State.supervising[staff] }
}
```

Equivalent relational form:

```
fun supervisors[s : StudentID] : set StaffID {
  State.supervising.s
}
```

Reason: The field `State.supervising` maps each staff member to the set of students they supervise:

```
StaffID -> set StudentID
```

So to find the supervisors of student *s*, we need all staff members *staff* such that *s* appears in `State.supervising[staff]`.

Schema: If a relation is stored as `Staff -> Student`, then looking up students for one staff member goes forward: `State.supervising[staff]`. Finding staff for one student goes backwards: `State.supervising.s`.

Q5. Alloy 3 - 6 marks

Continuing the university records scenario from Q3.

Write an Alloy predicate `inv` that describes the state invariant for the university records system. Add comments that describe your Alloy predicate. Your invariant should capture the constraints on the state mentioned in the description in Q3 that are not captured by the signature declarations.

For this question you need to carefully read the description in Q3 to first work out what these constraints are. You should document those constraints as English comments. You then need to logically encode them in your `inv` predicate. You are free to define extra functions to help you write your predicate, if you wish.

```
pred inv {  
  // FILL IN HERE  
}
```

Q5 Full-Mark Answer

One suitable answer is:

```
pred inv {  
  // Every student studying a Graduate Research degree has at least one supervisor.  
  all stu : StudentID |  
    State.student_degrees[stu] = GraduateResearch implies  
      some supervisors[stu]  
  
  // Only students studying a Graduate Research degree have supervisors.  
  all stu : StudentID |  
    some supervisors[stu] implies  
      State.student_degrees[stu] = GraduateResearch  
  
  // Only current Academic staff members can supervise students.  
  all staff : StaffID |  
    some State.supervising[staff] implies  
      State.staff_roles[staff] = Academic  
}
```

Equivalent without using the helper function from Q4:

```
pred inv {  
  // Every student studying a Graduate Research degree has at least one supervisor.  
  all stu : StudentID |  
    State.student_degrees[stu] = GraduateResearch implies  
      some staff : StaffID | stu in State.supervising[staff]  
  
  // Only students studying a Graduate Research degree have supervisors.  
  all staff : StaffID | all stu : State.supervising[staff] |  
    State.student_degrees[stu] = GraduateResearch  
  
  // Only current Academic staff members can supervise students.  
  all staff : StaffID |  
    some State.supervising[staff] implies  
      State.staff_roles[staff] = Academic  
}
```

Reason: The signature declarations already say each staff ID maps to at most one role and each student ID maps to at most one degree type. The invariant must add the cross-field rules: Graduate Research students need supervisors; supervised students must be Graduate Research students; and anyone supervising students must be an Academic staff member.

Schema: For an Alloy invariant, first separate what the signatures already enforce from what needs a cross-relation constraint. Here the invariant links `student_degrees`, `supervising`, and `staff_roles`.

Q6. Alloy 4 - 6 marks

Continuing the university records scenario from Q3.

Write the implementation for the following Alloy predicate `change_supervisors` that takes as its arguments a `StudentID` `stu` and a non-empty set of `StaffID` values `sup`s, and models the operation of changing the supervisors for the student identified by `stu` so that they are the supervisors identified by the IDs in `sup`s.

Your operation should include appropriate preconditions to ensure that the state invariant is maintained. Specifically, it should check that the student identified by `stu` is enrolled in a Graduate Research degree, and that all supervisors in `sup`s have the Academic role.

```
pred change_supervisors[stu : StudentID, sups : some StaffID] {
  // FILL IN HERE
}
```

Q6 Full-Mark Answer

```
pred change_supervisors[stu : StudentID, sups : some StaffID] {
  // The student whose supervisors are being changed must be enrolled
  // in a Graduate Research degree.
  State.student_degrees[stu] = GraduateResearch

  // Every new supervisor must be a current Academic staff member.
  all sup : sups |
    State.staff_roles[sup] = Academic

  // Replace exactly the supervisors of stu with sups.
  // Remove all old staff -> stu pairs, then add the new sups -> stu pairs.
  State.supervising' =
    State.supervising - (StaffID -> stu) + (sups -> stu)

  // Frame conditions: the operation does not change roles or degrees.
  State.staff_roles' = State.staff_roles
  State.student_degrees' = State.student_degrees
}
```

Equivalent update using the Q4 helper function:

```
State.supervising' =
  State.supervising - (supervisors[stu] -> stu) + (sups -> stu)
```

Reason: The operation changes the set of staff who supervise exactly one student, `stu`. Since `State.supervising` maps `StaffID -> StudentID`, old supervisors are removed by deleting the old pairs whose second component is `stu`, and the new pairs are added with `sup`s -> `stu`. The `sup`s argument is a set, so the Academic precondition must quantify over every member of `sup`s.

Schema: For a set-valued relation, use subtraction and union to replace the range entries for one target. Do not use `++` here unless you really want to override all tuples whose first component is in `sup`s.

Q7. Alloy 5 - 3 marks

Continuing the university records scenario from Q3.

Using the `assert t` command, write an Alloy assertion called `change_supervisors_works` that checks that whenever `change_supervisors[stu, sups]` occurs, for a `StudentID` `stu` and non-empty set of `StaffID` values `sup`s, then in the state afterwards it is the case that:

```
supervisors[stu] = sups

assert change_supervisors_works {
  // FILL IN HERE
}
```

Q7 Full-Mark Answer

```
assert change_supervisors_works {
  always all stu : StudentID, sups : some StaffID |
    change_supervisors[stu, sups] implies
      after supervisors[stu] = sups
}
```

Reason: The predicate `change_supervisors[stu, sups]` describes a transition from the current state to the next state. Therefore the assertion must check `supervisors[stu] = sups` in the next state, using `after`. Without `after`, the assertion would incorrectly check whether `stu` already has exactly those supervisors before the operation occurs.

Schema: For temporal Alloy assertions about operations, use:

```
always all args |
  operation[args] implies after postcondition
```

Q8. Alloy 6 - 2 marks

Continuing the university records scenario from Q3.

Consider the following check command.

```
check change_supervisors_works for 1..1 steps
```

Suppose Alloy performs this check and reports no counter-examples found. Explain what the bound for `1..1` steps on this command means, and what guarantees it does and does not provide.

If you believe the bound is sufficient to provide meaningful guarantees, explain why. If you believe it is not sufficient, then propose and justify a suitable bound that is no larger than necessary to provide those guarantees.

Your answer:

Q8 Full-Mark Answer

The bound ``for 1..1 steps`` means Alloy checks traces containing exactly one state. That gives no real state transition from a pre-state to a post-state.

This is not sufficient for `change_supervisors_works`, because the assertion is about what holds **after** `change_supervisors[stu, sups]` occurs. With only one state, Alloy cannot meaningfully check the effect of an operation on the next state. A report of "no counterexamples found" therefore gives only a vacuous or near-vacuous guarantee for this transition property.

A suitable smallest meaningful bound is:

```
check change_supervisors_works for 2..2 steps
```

or, less tightly:

```
check change_supervisors_works for 1..2 steps
```

The exact `2..2` steps bound is enough because the property checks a single-step postcondition: if the operation occurs in one state, the required property must hold in the next state. More than two states are not needed to check this immediate post-state property, although the usual Alloy caveat still applies: the guarantee is bounded by the chosen atom scopes.

Schema: If an assertion contains `after`, the trace must include a next state. One operation step needs two states: pre-state and post-state.

Q9. Hoare Logic 1 - 1 mark

The following program operates over mathematical, i.e. unbounded, integers where overflow cannot occur. It calculates an integer approximation to the square root of the integer `X`, placing the result in the integer variable `r`.

```
r := 0;
while (r+1)*(r+1) <= X do
  r := r + 1;
done
```

The postcondition for this program is:

```
r*r <= X and X < (r+1)*(r+1)
```

Will this postcondition be satisfied for all mathematical integers X ? Write down the weakest precondition that guarantees that this program will satisfy its postcondition when it terminates.

Your answer:

Q9 Full-Mark Answer

No, the postcondition is not satisfied for all mathematical integers X .

Weakest precondition:
 $X \geq 0$

Reason: If X is negative, the loop guard is false immediately because $(0 + 1) * (0 + 1) \leq X$ is false. The program terminates with $r = 0$, but the postcondition requires:

$$r * r \leq X$$

which becomes:

$$0 \leq X$$

This is false for negative X . If $X \geq 0$, then initially $r = 0$ gives $r * r \leq X$, and the loop increments r until $(r + 1) * (r + 1) > X$. At that point both parts of the postcondition hold:

$$\begin{aligned} r * r &\leq X \\ X &< (r + 1) * (r + 1) \end{aligned}$$

Boundary check: $X = 0$ is allowed. The loop does not execute, $r = 0$, and the postcondition $0 \leq 0$ and $0 < 1$ holds.

Schema: For a weakest precondition before a program, account for assignments before the loop. Since the program starts with $r := 0$, the weakest precondition should constrain X , not the old value of r .

Q10. Hoare Logic 2 - 4 marks

Continuing with the square root program from Q9.

Write down a suitable loop invariant for the while-loop of this program. Your invariant should:

- ensure that the program meets its postcondition when the loop terminates,
- be satisfied when the program reaches the loop, i.e. just after the assignment $r := 0$; and
- be preserved by each iteration of the loop body.

Using English, briefly justify why your invariant meets these three conditions, in no more than a few sentences for each.

$$I = k \text{ in } 0 \dots (r-1) \text{ and } ((k+1)*(k+1) \leq X) \text{ and } (r*r \leq X \text{ and } X < (r+1) * (r+1))$$

Reasoning:

>

Q10 Full-Mark Answer

$$I = r * r \leq X$$

A slightly stronger acceptable invariant is:

$$I = r \geq 0 \text{ and } r * r \leq X$$

1. It implies the postcondition at loop exit.

When the loop terminates, the guard is false:

$$\text{not } ((r+1)*(r+1) \leq X)$$

so:

$$X < (r+1)*(r+1)$$

Combining this with the invariant $r*r \leq X$ gives the required postcondition:

$$r*r \leq X \text{ and } X < (r+1)*(r+1)$$

2. It holds when the loop is first reached.

From Q9, the precondition is:

$$X \geq 0$$

After:

$$r := 0$$

we have:

$$r*r = 0$$

so $r*r \leq X$ holds because $0 \leq X$.

3. It is preserved by each loop iteration.

Assume the invariant holds and the guard is true:

$$\begin{aligned} r*r &\leq X \\ (r+1)*(r+1) &\leq X \end{aligned}$$

The loop body executes:

$$r := r + 1$$

After this assignment, the new value of $r*r$ is the old value of $(r+1)*(r+1)$, which the guard already told us is at most X . Therefore the invariant $r*r \leq X$ is preserved.

Schema: Do not put the whole final postcondition into the invariant. The second half of the postcondition, $X < (r+1)*(r+1)$, comes from the negated loop guard at exit, not from the invariant itself.

Q11. Hoare Logic 3 - 7 marks

Continuing with the square root program from Q9.

Use the rules of Hoare logic to prove that this program meets its specification, i.e. satisfies its postcondition assuming the precondition you defined for Q9.

Proof:

>

Q11 Full-Mark Answer

Using the Q9 precondition:

$$X \geq 0$$

and the Q10 loop invariant:

$$I = r*r \leq X$$

we can prove the program as follows.

$$\{X \geq 0\}$$

```

{0*0 <= X}
r := 0;
{r*r <= X}

while (r+1)*(r+1) <= X do
  {r*r <= X and (r+1)*(r+1) <= X}
  {(r+1)*(r+1) <= X}
  r := r + 1;
  {r*r <= X}
done

{r*r <= X and not ((r+1)*(r+1) <= X)}
{r*r <= X and X < (r+1)*(r+1)}

```

Justification:

The assignment $r := 0$ establishes the invariant because the precondition $X \geq 0$ implies $0*0 \leq X$.

For preservation, assume the invariant and the loop guard:

$$r*r \leq X \text{ and } (r+1)*(r+1) \leq X$$

To prove the invariant after $r := r + 1$, the assignment rule says we need:

$$(r+1)*(r+1) \leq X$$

which is exactly the loop guard. So the body preserves $r*r \leq X$.

At loop exit, the while rule gives:

$$r*r \leq X \text{ and not } ((r+1)*(r+1) \leq X)$$

Over mathematical integers, not $((r+1)*(r+1) \leq X)$ is equivalent to:

$$X < (r+1)*(r+1)$$

Therefore the required postcondition holds:

$$r*r \leq X \text{ and } X < (r+1)*(r+1)$$

Schema: A Hoare proof for a while loop uses the rule:

```

if {I and B} S {I}, then
{I} while B do S done {I and not B}

```

Then use the consequence rule to turn I and not B into the required postcondition.

Q12. Hoare Logic 4 - 4 marks

Bob has proposed that the following rule can be used to reason about if-statements, i.e. conditionals:

$$\frac{\{P \text{ and } B\} S1 \{Q\} \quad \{P \text{ and not } B\} S2 \{Q\}}{\{P\} \text{ if } B \text{ then } S1 \text{ else } S2 \{Q\}}$$

This rule says that for some precondition P and postcondition Q , to prove that:

$$\{P\} \text{ if } B \text{ then } S1 \text{ else } S2 \{Q\}$$

holds, it is sufficient to prove that both:

$$\{P \text{ and } B\} S1 \{Q\}$$

and:

$$\{P \text{ and not } B\} S2 \{Q\}$$

hold, where and means conjunction and not means logical negation.

Show that Bob's rule is valid. That is, assuming that both $\{P \text{ and } B\} \vdash S1 \{Q\}$ and $\{P \text{ and not } B\} \vdash S2 \{Q\}$ hold, prove that $\{P\} \text{ if } B \text{ then } S1 \text{ else } S2 \{Q\}$ holds.

Your answer:

Q12 Full-Mark Answer

Assume both premises hold:

1. $\{P \text{ and } B\} \vdash S1 \{Q\}$
2. $\{P \text{ and not } B\} \vdash S2 \{Q\}$

We must show:

$\{P\} \text{ if } B \text{ then } S1 \text{ else } S2 \{Q\}$

Consider any initial state satisfying P.

There are two cases.

Case 1: B is true.

Since the initial state satisfies P and B, it satisfies:

$P \text{ and } B$

The conditional executes the then-branch S1. By premise 1:

$\{P \text{ and } B\} \vdash S1 \{Q\}$

so after executing S1, Q holds.

Case 2: B is false.

Since the initial state satisfies P and not B, it satisfies:

$P \text{ and not } B$

The conditional executes the else-branch S2. By premise 2:

$\{P \text{ and not } B\} \vdash S2 \{Q\}$

so after executing S2, Q holds.

In all possible cases, executing the conditional from a state satisfying P leads to a state satisfying Q. Therefore:

$\{P\} \text{ if } B \text{ then } S1 \text{ else } S2 \{Q\}$

is valid.

Schema: To prove a conditional rule, split on the Boolean guard. The then-branch gets $P \text{ and } B$; the else-branch gets $P \text{ and not } B$. If both branches establish the same postcondition Q, then the whole conditional establishes Q.

Q13. Pointers 1 - 2 marks

Consider the trivial C program that takes two pointers p and q that each point to int values. The program simply stores the value 5 into the memory location that p points to, and then stores the value 6 into the memory location that q points to.

```
void prog(int *p, int *q) {
    *p = 5;
    *q = 6;
}
```

After this code executes, the value stored in the location that p points to will be:

Answer:

because:

Reason:

Q13 Full-Mark Answer

Answer:

Impossible to say; it could be 5 or 6.

Reason:

The pointers p and q might alias, meaning they might point to the same heap location. If p and q point to different locations, then $*p = 5$ leaves the location pointed to by p containing 5, and $*q = 6$ changes only q 's location. But if p and q point to the same location, then $*p = 5$ first writes 5, and then $*q = 6$ writes 6 to that same location. In that aliasing case, the value stored in the location that p points to is 6.

Schema: With pointers, different variable names do not imply different heap locations. Unless a precondition rules out aliasing, an update through q may also change what is seen through p .

Q14. Pointers 2 - 2 marks

Continuing with the pointer C program from Q13.

Answer the following questions.

The separation logic assertion:

$p \mapsto 5$ and $q \mapsto 6$

implies that:

$p \mapsto 5 * q \mapsto 6$

true / false:

For the C program prog above from Q13, the weakest separation logic precondition required to guarantee the postcondition:

$p \mapsto 5 * q \mapsto 6$

is which of the following?

- A. There is no such weakest precondition.
- B. $p \mapsto 5 * q \mapsto 6$
- C. $\exists u, v. p \mapsto u * q \mapsto v$

Choice:

Q14 Full-Mark Answer

First part:

False.

Second part:

C. $\exists u, v. p \mapsto u * q \mapsto v$

Reason for first part: Ordinary logical and does not imply separation. The assertion:

$p \mapsto 5$ and $q \mapsto 6$

says both heap facts hold, but it does not itself provide the separating conjunction fact that the heap cells are disjoint. The assertion:

$p \mapsto 5 * q \mapsto 6$

is stronger because $*$ says the two points-to assertions hold on disjoint parts of the heap.

Reason for second part: To guarantee the postcondition:

```
p |-> 5 * q |-> 6
```

the program must start with p and q pointing to two separate valid heap cells. Their old contents do not matter, because the program overwrites them with 5 and 6. Therefore the weakest separation-logic precondition is:

```
exists u, v . p |-> u * q |-> v
```

Schema: and combines facts; $*$ combines facts and also states that the heap portions are separate.

Q15. Pointers 3 - 5 marks

Continuing with the pointer C program from Q13.

Alice has translated this program into Ada and has added SPARK annotations to specify a precondition and postcondition for it. Suppose she runs the SPARK Prover on this code. Will it report that it meets its specification? Justify your answer in no more than a paragraph of text.

```
type IntPtr is access Integer;

procedure Prog(P : in IntPtr; Q : in IntPtr) with
  Pre => (P /= null and Q /= null),
  Post => (P.all = 5 and Q.all = 6)
is
begin
  P.all := 5;
  Q.all := 6;
end Prog;
```

Your answer:

Q15 Full-Mark Answer

No. The SPARK Prover will not be able to prove that this procedure satisfies the postcondition from the given precondition.

Reason: The precondition says only that P and Q are non-null:

```
P /= null and Q /= null
```

It does not say that P and Q point to different memory locations. If P and Q alias, then both access values point to the same integer. The first assignment:

```
P.all := 5;
```

sets that shared cell to 5, but the second assignment:

```
Q.all := 6;
```

then sets the same cell to 6. In that case, at the end of the procedure, $Q.all = 6$ is true but $P.all = 5$ is false. To make the specification provable, the precondition would need to rule out aliasing, for example:

```
Pre => (P /= null and Q /= null and P /= Q)
```

Schema: In SPARK contracts over access values, non-null is not the same as non-aliasing. If a postcondition needs two pointed-to values to remain independent, the precondition must rule out $P = Q$.

Q16. Safe Languages 1 - 5 marks

Consider the following simple C function. Suppose this function is used to transform floating-point data by increasing each value by 2. For instance, if given the input 5.5, this function is supposed to produce the output value 7.5.

```
float convert(float f) {
  int a = (2 + f);
  return a;
}
```

Unfortunately, it does not work correctly. The output it produces when given the input 5.5 is 7.0, instead of 7.5. Similarly, when given the input 2.3 it produces 4.0 instead of 4.3.

Re-write this function in Ada. Explain what would happen if you tried to compile this function and how the compiler assists in locating the problem or problems with the function.

```
-- Your Ada rewrite
function Convert(f : in Float) return Float
  Float a;
is
begin
  a := To_FLOAT(2) + f
end Convert
```

Explanation:

>

Q16 Full-Mark Answer

Direct Ada rewrite of the buggy C function:

```
function Convert(F : Float) return Float is
  A : Integer := 2 + F;
begin
  return A;
end Convert;
```

This would not compile in Ada. The compiler would report type errors because the code mixes Integer and Float values in places where Ada does not allow implicit conversions.

There are two main problems:

1. A is declared as an Integer, but 2 + F is a floating-point expression because F is a Float. Ada will not silently convert the fractional result to an integer.
2. The function is declared to return Float, but return A; attempts to return an Integer.

The compiler helps by pointing to the exact lines where the type mismatches occur and by saying which type was expected and which type was found. This is the safety benefit: the truncation that C allowed silently is caught at compile time in Ada.

A corrected Ada implementation would avoid the unnecessary integer variable:

```
function Convert(F : Float) return Float is
begin
  return F + 2.0;
end Convert;
```

OR:

```
function Convert(F : Float) return Float is
  A : Float := F + 2.0;
begin
  return A;
end Convert;
```

Schema: In C, assigning a floating-point expression to an int may silently drop the fractional part. In Ada, strong static typing forces the programmer to make such conversions explicit, so accidental truncation is detected during compilation.

Q17. Safe Languages 2 - 10 marks

A fictitious study was carried out in which 10,000 C programs were downloaded from GitHub. Each program was re-written in Ada. Fuzzing was then applied to both sets of programs, without applying any memory sanitizers or undefined-behaviour sanitizers to the C programs.

It was found that, when fuzzed, the Ada programs were more likely to abruptly terminate, such as by raising `CONSTRAINT_ERRORS` relating to integer overflow and array indexes being out of bounds. In contrast, the C programs were less likely to abruptly terminate.

In addition, the study found that, on average, it takes longer to write an equivalent program in Ada than in C, because of Ada's strong, static type system.

The authors of the study conclude therefore that C should be preferred for writing high-integrity software because C programs are less likely to abruptly terminate than Ada programs, and because they can be written more quickly and therefore more cheaply.

Do you agree with the authors of the study? Justify your answer in no more than 3 paragraphs. Bullet point answers are OK.

Your answer:

Q17 Full-Mark Answer

No, I do not agree with the authors' conclusion. The fact that the Ada programs terminate more often under fuzzing does not by itself show that Ada is worse for high-integrity software. Ada has well-defined runtime checks for faults such as integer overflow and array-index errors, so it may stop at the point where an invalid operation is detected. In C, the same kind of fault may lead to undefined behaviour, memory corruption, or silently wrong results, especially since the study did not use memory sanitizers or undefined-behaviour sanitizers. A C program that does not terminate may simply be continuing in an unsafe or incorrect state.

For high-integrity software, visible failure is often preferable to silent failure. An abrupt `CONSTRAINT_ERROR` is still a defect that must be fixed, but it gives precise evidence of a violated assumption and helps locate the problem. Ada's strong static type system and runtime checks are intended to make faults easier to detect, reason about, and remove. They also reduce classes of errors that C programmers must manage manually.

I also do not agree that faster initial development implies lower total cost or better suitability for high-integrity systems. Ada may take longer to write because the programmer must be more explicit about types, ranges, and interfaces, but that upfront effort can reduce later debugging, testing, verification, certification, and maintenance effort. In high-integrity software, correctness, predictability, analyzability, and evidence for safety are more important than minimising initial coding time. The study's observations may show a tradeoff, but they do not justify preferring C for high-integrity software.

Schema: Do not treat "program did not crash" as the same as "program was safe." Safe languages often turn hidden faults into visible failures, which is valuable for high-integrity engineering.